1.Introduction to C++

1. What are the key differences between Procedural Programming and Object Oriented Programming (OOP)?

POP: Procedural Programming

- In POP, the program is divided into functions. It follows a top-down approach, meaning we start from the main problem and break it into smaller functions.
- The main focus is on procedures or functions, not on data.
- Data is often shared globally, so it is less secure.
- POP languages include C.

OOP: Object-Oriented Programming

- In OOP, the program is divided into objects and classes.
- t follows a bottom-up approach, where small objects are created first and then combined.
- The main focus is on data along with functions.
- Data is protected using encapsulation, so it is more secure.
- OOP also supports features like inheritance and polymorphism, which help in code reuse and flexibility.
- OOP languages include C++ and Java.

2. List and explain the main advantages of OOP over POP.

Advantages of OOP over POP

1. Data Security (Encapsulation)

- Data is hidden inside classes
- Access only through methods

2. Code Reusability (Inheritance)

- One class can reuse properties of another
- Reduces code duplication

3. Flexibility (Polymorphism)

- Same function behaves differently
- Example: function overloading

4. Modularity

- Program divided into small objects
- Easy to manage and debug

5. Real-world Modeling

- Objects represent real-world entities
- Makes design easier

6. Easy Maintenance

- Changes in one class don't affect others

3. Explain the steps involved in setting up a C++ development environment.

Install IDE (Optional but Recommended)

- IDE examples:
    - Code::Blocks
    - Dev C++
    - Visual Studio

Configure Environment

- Set compiler path (if needed)
- Check installation using command:

  g++ --version

Write First Program

```
#include<iostream>
using namespace std;

int main() {
   cout << "Hello World";
   return 0;
}
```

Compile & Run

- Compile:

  g++ file.cpp

- Run:

  ./a.out

4. What are the main input/output operations in C++? Provide examples.

Cout:

```
#include<iostream>
using namespace std;

int main() {
   cout << "Hello World";
   return 0;
}
```

Cin:

```
#include<iostream>
using namespace std;

int main() {
```

```
    int a;
    cin >> a;
    cout << "Value: " << a;
    return 0;
}
```

Multiple Inputs

```
int a, b;
cin >> a >> b;
```

Formatted Output

```
cout << "Sum = " << a + b;
```

2. Variables, Data Types, and Operators

1. What are the different data types available in C++? Explain with examples.

1. Basic (Primitive) Data Types

- int → integers

  int a = 10;

- float → decimal numbers

  float b = 5.5;

- double → large decimal numbers

  double c = 10.12345;

- char → single character

  char d = 'A';

- bool → true/false

  bool e = true;

2. Derived Data Types

- Array

  int arr[3] = {1, 2, 3};

- Pointer

  int *p;

3. User-defined Data Types

- struct
- union
- enum
- class

Ex:

```
struct Student {
   int id;
};
```

2. Explain the difference between implicit and explicit type conversion in C++.

Implicit Conversion (Automatic)

- Done automatically by compiler
- Smaller type → larger type

Example:

```
int a = 10;
float b = a;   // int → float (automatic)
```

Explicit Conversion (Type Casting)

- Done manually by programmer

- Use casting operator

Example:

```
float a = 10.5;
int b = (int)a;   // float → int (manual)
```

3. What are the different types of operators in C++? Provide examples of each.

1. Arithmetic Operators

+ - * / %

```
int c = a + b;
```

2. Relational Operators

== != > < >= <=

```
if(a > b)
```

3. Logical Operators

&& || !

```
if(a > 0 && b > 0)
```

4. Assignment Operators

= += -= *= /=

```
a += 5;
```

5. Increment/Decrement

++ --

a++;

6. Bitwise Operators

& | ^ ~ << >>

7. Conditional (Ternary) Operator

int max = (a > b) ? a : b;

4. Explain the purpose and use of constants and literals in C++.

Constants

- Fixed value (cannot be changed)

Example:

const int a = 10;

Value cannot be modified later

Literals

- Actual values written in code

Types:

- Integer literal → 10
- Float literal → 5.5
- Character literal → 'A'
- String literal → "Hello"

Example:

```
int a = 10;   // 10 is literal
```

3. Control Flow Statements

1. What are conditional statements in C++? Explain the if-else and switch statements.

if-else statement

- Executes code based on condition (true/false)

Syntax:

```
if(condition) {
   // code if true
} else {
   // code if false
}
```

switch statement

- Used when multiple choices are available

Syntax:

```
switch(value) {
   case 1:
      // code
      break;
   case 2:
      // code
      break;
   default:
      // code
}
```

2. What is the difference between for, while, and do-while loops in C++?

for loop

- Used when number of iterations is known
- Initialization, condition, increment in one line

```
for(int i = 1; i <= 5; i++) {
   cout << i;
}
```

while loop

- Used when condition is checked first
- May execute 0 times

```
int i = 1;
while(i <= 5) {
   cout << i;
   i++;
}
```

do-while loop

- Condition checked after execution
- Executes at least 1 time

```
int i = 1;
do {
   cout << i;
   i++;
} while(i <= 5);
```

3. How are break and continue statements used in loops? Provide examples.

break

- Used to exit loop immediately

```
for(int i = 1; i <= 5; i++) {
   if(i == 3) break;
   cout << i;
}
```

👉 Output: 1 2

continue

- Skips current iteration

```
for(int i = 1; i <= 5; i++) {
   if(i == 3) continue;
   cout << i;
}
```

4. Explain nested control structures with an example.

Nested means one control structure inside another (loop inside loop or if inside if).

Ex (nested loop):

```
for(int i = 1; i <= 3; i++) {
   for(int j = 1; j <= 2; j++) {
      cout << i << " " << j << endl;
   }
}
```